COMPUTER SCIENCE + COMPUTER ENGINEERING

06

Start Package 06

Readiness, setup and the first session

Begin with the CPU route · Choose one GPU route later · Preserve evidence

PACKAGE 06 / EXPANDED VOLUME 05 / SEPTEMBER 28, 2026
Self-study: derive, implement, test and explain.

Start Package 06

Open the right files

Begin with STUDENT_HANDBOOK.pdf or the offline COURSE.html. Keep LAB_WORKBOOK.pdf beside it for the lesson procedure and coding contract. Open ANSWER_KEY.pdf only after an attempt. CAPSTONE_SPEC.pdf is the finish line; ENGINE_REFERENCE.pdf contains formulas, conventions, source-file mapping and platform notes. Each module is also available as a smaller PDF in modules/.

This package is Volume 05, not Volume 06. Preserve the existing lesson IDs in your repository and evidence log. The source/ folder is editable; templates/PROGRESS.csv has one row per lesson.

First session: 60-90 minutes

1. Extract the entire ZIP, keep its folder layout and open a terminal in Game_Engineering_Package_06.
2. Read the handbook introduction and Lesson 21.1. Write the coordinate/color contract before running anything.
3. Check the interpreter with `python --version` (or `py -3 --version` on Windows). Python 3.11+ is the selected syntax baseline; the included actual run used Python 3.13.5 on Linux.
4. Run the reference tests below, then run the starter test for 21.1 and observe its intentional failure.
5. Implement the image-size/stride function in the starter, rerun its tests and add a new padded-row case. Continue to the original image-generation lab; a size helper alone does not complete that lab.
6. Answer the five questions without the key. Record the input, expected bytes, observed result and one unresolved assumption.

```
python tests/test_exercises.py --track reference
python code/python/run_examples.py
python tests/test_exercises.py --track starter --lesson 21.1
```

The starter raises NotImplementedError by design. Do not point your grading command at --track reference after editing the starter. The six table cases per lesson are a starting test set; add tests derived from your own implementation assumptions.

C++ build route

Use a C++20 compiler and CMake 3.20 or later. No graphics SDK or numerical package is needed for the CPU reference. The following syntax is for a terminal at the package root; executable locations can differ by generator.

```
cmake -S code/cpp -B work/cpp -DCMAKE_BUILD_TYPE=Release
cmake --build work/cpp --config Release
ctest --test-dir work/cpp -C Release --output-on-failure
./work/cpp/cpu_renderer --output work/reference.ppm
./work/cpp/cpu_renderer --affine --output work/affine.ppm
```

For a Windows multi-configuration generator, use the actual `work/cpp/Release/cpu_renderer.exe` path. The supplied Linux run does not establish Windows or macOS validation. The saved PNG previews in evidence/ are immediately viewable even without a PPM viewer.

Run `python validate_all.py` to repeat all CPU checks in work/validation. It performs tests, C++ build, cooking, mesh loading/rendering and a BVH comparison. `--skip-cpp` is available for Python-only work, with C++ explicitly marked skipped. This runner never launches or tests a GPU.

GPU route: choose one, do not assume one

For Module 22, select Vulkan, Direct3D 12 or Metal according to available hardware and platform. The source-only WebGPU bridge is an additional introduction to resources, command recording, binding, compute/readback and reuse; it does not implement native barrier rules for you.

```
python -m http.server 8000 --bind 127.0.0.1 --directory code/webgpu
```

Open `http://localhost:8000` in a compatible browser. Record the actual browser and adapter results, not expected support. The page reports bounded readback for N=0,1,63,64,65, then draws a transformed cube. Resize and stop it, retain validation output and complete the remaining workbook cases. The package only syntax-checked this JavaScript; hardware execution is pending.

The localhost server exposes only the selected demo directory and only on loopback. Stop it with Ctrl+C. No paid account, external package download or GPU purchase is required to begin the CPU lessons.

Readiness check: eight items

These are placement prompts, not a validated admissions test or part of the 100 lesson questions. Use scratch paper before searching.

1. Divide clip x=2 by w=4. What is the normalized x coordinate?
2. How many bytes are in a tightly packed 320-by-180 RGBA8 image?
3. Why does a copied raw pointer not extend the pointed-to object's lifetime?
4. For barycentric weights 0.2, 0.3 and 0.5, what is their sum?
5. With a 10 ms fixed step, how many full updates fit into 25 ms and what fraction remains?
6. Does returning from GPU queue submission prove the buffer can be overwritten?
7. Is a normal vector texture normally decoded with an sRGB color transfer function?
8. What should a load operation do to the current valid world when candidate parsing fails?

Repair algebra/vector gaps using Volume 02, lifetime/data-structure gaps using Volume 03 and concurrency/persistence/build gaps using Volume 04. Use templates/PREREQUISITE_REPAIR.md and solve a new example after reviewing.

Completion and evidence

Suggested local mastery threshold: at least 4 of 5 lesson answers independently explained, every stated lab pass criterion met, a new transfer test passed and any safety/correctness-critical misconception repaired. This is a self-study review policy, not a substitute for substantive work. A learner who scores 5/5 but cannot implement or explain the lab should repeat the lab.

Keep GPU tests marked NOT RUN when no compatible device is available. Keep Gate E reviewer signoff NOT PERFORMED until another person builds and runs the integrated engine from the documented clean source. Automated authoring checks do not satisfy that independent review.